



Campusmonk

DSA Black Book

Campusmonk DSA Black Book

The Monk Way – Where DSA Meets Success

**Topic covered: Maths, Arrays, Strings, LL , Stack ,
Queues**

- Maths (all imp concepts)
- Arrays (all important algos + hashing + 2 pointer + sliding window)
- Strings (All Important pattern)
- Story based (TCS, Wipro, Accenture ...) PYQ
- Linked -Lists
- Stack & queues
- Greedy approach (Question practice)



1.) Count all Digits of a Number

[Link](#)

You are given an integer **n**. You need to **return** the **number** of digits in the number.

The number will have **no** leading zeroes, except when the number is **0** itself.

Examples:

Input: $n = 4$

Output: 1

Explanation: There is only 1 digit in 4.

Input: $n = 14$

Output: 2

Explanation: There are 2 digits in 14.

2.) Count number of odd digits in a number

You are given an integer **n**. You need to **return** the **number** of **odd** digits present in the number.

The number will have **no** leading zeroes, except when the number is **0** itself.

Examples:

Input: $n = 5$

Output: 1

Explanation: 5 is an odd digit.

Input: $n = 25$

Output: 1

Explanation: The only odd digit in 25 is 5.



3.) Reverse a number [Link](#)

You are given an integer n. Return the integer formed by placing the digits of n in reverse order.

Examples:

Input: n = 25

Output: 52

Explanation: Reverse of 25 is 52.

Input: n = 123

Output: 321

Explanation: Reverse of 123 is 321.

4.) Palindrome Number [Link](#)

You are given an integer n. You need to check whether the number is a palindrome number or not. Return true if it's a palindrome number, otherwise return false.

A palindrome number is a number which reads the same both left to right and right to left.

Examples:

Input: n = 121

Output: true

Explanation: When read from left to right : 121.

When read from right to left : 121.

Input: n = 123



Campusmonk

Output: false

Explanation: When read from left to right : 123.

When read from right to left : 321.

5.) Return the Largest Digit in a Number

You are given an integer n. Return the largest digit present in the number.

Examples:

Input: n = 25

Output: 5

Explanation: The largest digit in 25 is 5.

Input: n = 99

Output: 9

Explanation: The largest digit in 99 is 9.

6.) Divisors of a Number [Link](#)

You are given an integer n. You need to find all the divisors of n. Return all the divisors of n as an array or list in a sorted order.

A number which completely divides another number is called it's divisor.

Examples:

Input: n = 6

Output = [1, 2, 3, 6]

Explanation: The divisors of 6 are 1, 2, 3, 6.



Campusmonk

Input: $n = 8$

Output: [1, 2, 4, 8]

Explanation: The divisors of 8 are 1, 2, 4, 8.

7.) Factorial of a given number [Link](#)

You are given an integer n . Return the value of $n!$ or n factorial.

Factorial of a number is the product of all positive integers less than or equal to that number.

Examples:

Input: $n = 2$

Output: 2

Explanation: $2! = 1 * 2 = 2$.

Input: $n = 0$

Output: 1

Explanation: $0!$ is defined as 1.

8.) Check if the Number is Armstrong [Link](#)

You are given an integer n . You need to check whether it is an armstrong number or not. Return true if it is an armstrong number, otherwise return false.

An Armstrong number is a number which is equal to the sum of the digits of the number, raised to the power of the number of digits.



Campusmonk

Examples:

Input: n = 153

Output: true

Explanation: Number of digits : 3.

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.$$

Therefore, it is an Armstrong number.

Input: n = 12

Output: false

Explanation: Number of digits : 2.

$$1^2 + 2^2 = 1 + 4 = 5.$$

Therefore, it is not an Armstrong number.

9.) Check for Perfect Number [Link](#)

You are given an integer n. You need to check if the number is a perfect number or not. Return true if it is a perfect number, otherwise, return false.

A perfect number is a number whose proper divisors add up to the number itself.

Examples:

Input: n = 6

Output: true

Explanation: Proper divisors of 6 are 1, 2, 3.

$$1 + 2 + 3 = 6.$$

Input: n = 4

Output: false

Explanation: Proper divisors of 4 are 1, 2.

$$1 + 2 = 3.$$



10.) Check for Prime Number [Link](#)

You are given an integer n . You need to check if the number is prime or not. Return true if it is a prime number, otherwise return false.

A prime number is a number which has no divisors except 1 and itself.

Examples:

Input: $n = 5$

Output: true

Explanation: The only divisors of 5 are 1 and 5, So the number 5 is prime.

Input: $n = 8$

Output: false

Explanation: The divisors of 8 are 1, 2, 4, 8, thus it is not a prime number.

11.) Count of Prime Numbers till N [Link](#)

You are given an integer n . You need to find out the number of prime numbers in the range $[1, n]$ (inclusive). Return the number of prime numbers in the range.

A prime number is a number which has no divisors except, 1 and itself.

Examples:

Input: $n = 6$

Output: 3

Explanation: Prime numbers in the range $[1, 6]$ are 2, 3, 5.

Input: $n = 10$

Output: 4



Explanation: Prime numbers in the range [1, 10] are 2, 3, 5, 7.

12.) GCD of Two Numbers [Link](#)

You are given two integers $n1$ and $n2$. You need find the Greatest Common Divisor (GCD) of the two given numbers. Return the GCD of the two numbers.

The Greatest Common Divisor (GCD) of two integers is the largest positive integer that divides both of the integers.

Examples:

Input: $n1 = 4, n2 = 6$

Output: 2

Explanation: Divisors of $n1 = 1, 2, 4$, Divisors of $n2 = 1, 2, 3, 6$

Greatest Common divisor = 2.

Input: $n1 = 9, n2 = 8$

Output: 1

Explanation: Divisors of $n1 = 1, 3, 9$ Divisors of $n2 = 1, 2, 4, 8$.

Greatest Common divisor = 1.

13.) LCM of two numbers [Link](#)

You are given two integers $n1$ and $n2$. You need find the Lowest Common Multiple (LCM) of the two given numbers. Return the LCM of the two numbers.

The Lowest Common Multiple (LCM) of two integers is the lowest positive integer that is divisible by both the integers.

Examples:

Input: $n1 = 4, n2 = 6$



Campusmonk

Output: 12

Explanation: $4 * 3 = 12$, $6 * 2 = 12$.

12 is the lowest integer that is divisible both 4 and 6.

Input: $n1 = 3$, $n2 = 5$

Output: 15

Explanation: $3 * 5 = 15$, $5 * 3 = 15$.

15 is the lowest integer that is divisible both 3 and 5.

14.) Print all primes till N [Link](#)

You are given an integer n .

Print all the prime numbers till n (including n).

A prime number is a number that has only two divisor's 1 and the number itself.

Examples:

Input : $n = 7$

Output : [2, 3, 5, 7]

Explanation : The number 2 has only two divisors 1 and 2.

The number 3 has only two divisors 1 and 3.

The number 5 has only two divisors 1 and 5.

The number 7 has only two divisors 1 and 7.

Input : $n = 2$

Output : [2]

Explanation : There is only one number 2 that is a prime till 2.

15.) Prime factorisation of a Number [Link](#)

You are given an integer array queries of length n .

Return the prime factorization of each number in array queries in sorted order.



Campusmonk

Examples:

Input : queries = [2, 3, 4, 5, 6]

Output : [[2], [3], [2, 2], [5], [2, 3]]

Explanation : The values 2, 3, 5 are itself prime numbers.

The prime factorization of 4 will be --> $2 * 2$.

The prime factorization of 6 will be --> $2 * 3$.

Input : queries = [7, 12, 18]

Output : [[7], [2, 2, 3], [2, 3, 3]]

Explanation : The value 7 itself is a prime number.

The prime factorization of 12 will be --> $2 * 2 * 3$.

The prime factorization of 18 will be --> $2 * 3 * 3$.

16.) Count primes in range L to R

You are given an 2D array queries of dimension $n * 2$.

The queries[i] represents a range from queries[i][0] to queries[i][1] (include the end points).

Return the count of prime numbers present in between each range in queries array.

Examples:

Input : queries = [[2, 5], [4, 7]]

Output : [3, 2]

Explanation : The range 2 to 5 contains three prime numbers 2, 3, 5.

The range 4 to 7 contains two prime numbers 5, 7.

Input : queries = [[1, 7], [3, 7]]

Output : [4, 3]

Explanation : The range 1 to 7 contains four prime numbers 2, 3, 5, 7.

The range 3 to 7 contains three prime numbers 3, 5, 7.

17.) Sum of Array Elements [Link](#)



Campusmonk

Given an array `arr` of size `n`, the task is to find the sum of all the elements in the array.

Examples:

Input: `n=5, arr = [1,2,3,4,5]`

Output: 15

Explanation: Sum of all the elements is $1+2+3+4+5 = 15$

Input: `n=6, arr = [1,2,1,1,5,1]`

Output: 11

Explanation: Sum of all the elements is $1+2+1+1+5+1 = 11$

18.) Count of odd numbers in array

Given an array of `n` elements. The task is to return the count of the number of odd numbers in the array.

Examples:

Input: `n=5, array = [1,2,3,4,5]`

Output: 3

Explanation: The three odd elements are (1,3,5).

Input: `n=6, array = [1,2,1,1,5,1]`

Output: 5

Explanation: The five odd elements are one 5 and four 1's.



19.) Check if the Array is Sorted I [Link](#)

Given an array arr of size n, the task is to check if the given array is sorted in (ascending / Increasing / Non-decreasing) order. If the array is sorted then return True, else return False.

Examples:

Input: n = 5, arr = [1,2,3,4,5]

Output: True

Explanation: The given array is sorted i.e Every element in the array is smaller than or equals to its next values, So the answer is True.

Input: n = 5, arr = [5,4,6,7,8]

Output: False

20.) Reverse an array [Link](#)

Given an array arr of n elements. The task is to reverse the given array. The reversal of array should be in place.

Examples:

Input: n=5, arr = [1,2,3,4,5]

Output: [5,4,3,2,1]

Explanation: The reverse of the array [1,2,3,4,5] is [5,4,3,2,1]

Input: n=6, arr = [1,2,1,1,5,1]

Output: [1,5,1,1,2,1]

Explanation: The reverse of the array [1,2,1,1,5,1] is [1,5,1,1,2,1].



21.) Highest Occurring Element in an Array [Link](#)

Given an array of n integers, find the most frequent element in it i.e., the element that occurs the maximum number of times. If there are multiple elements that appear a maximum number of times, find the smallest of them.

Examples:

Input: arr = [1, 2, 2, 3, 3, 3]

Output: 3

Explanation: The number 3 appears the most (3 times). It is the most frequent element.

Input: arr = [4, 4, 5, 5, 6]

Output: 4

Explanation: Both 4 and 5 appear twice, but 4 is smaller. So, 4 is the most frequent element.

22.) Second Highest Occurring Element

Given an array of n integers, find the second most frequent element in it. If there are multiple elements that appear a maximum number of times, find the smallest of them. If second most frequent element does not exist return -1.

Examples:

Input: arr = [1, 2, 2, 3, 3, 3]

Output: 2

Explanation: The number 2 appears the second most (2 times) and number 3 appears the most (3 times).



Campusmonk

Input: arr = [4, 4, 5, 5, 6, 7]

Output: 6

Explanation: Both 6 and 7 appear second most times, but 6 is smaller.

23.) Linear Search

[Link](#)

Given an array of integers nums and an integer target, find the smallest index (0 based indexing) where the target appears in the array. If the target is not found in the array, return -1

Examples:

Input: nums = [2, 3, 4, 5, 3], target = 3

Output: 1

Explanation: The first occurrence of 3 in nums is at index 1

Input: nums = [2, -4, 4, 0, 10], target = 6

Output: -1

Explanation: The value 6 does not occur in the array, hence output is -1

24.) Largest Element

[Link](#)

Given an array of integers nums, return the value of the largest element in the array

Examples:

Input: nums = [3, 3, 6, 1]

Output: 6

Explanation: The largest element in array is 6

Input: nums = [3, 3, 0, 99, -40]



Campusmonk

Output: 99

Explanation: The largest element in array is 99

25.) Second Largest Element [Link](#)

Given an array of integers nums, return the second-largest element in the array. If the second-largest element does not exist, return -1.

Examples:

Input: nums = [8, 8, 7, 6, 5]

Output: 7

Explanation: The largest value in nums is 8, the second largest is 7

Input: nums = [10, 10, 10, 10, 10]

Output: -1

Explanation: The only value in nums is 10, so there is no second largest value, thus -1 is returned

26.) Maximum Consecutive Ones [Link](#)

Given a binary array nums, return the maximum number of consecutive 1s in the array.

A binary array is an array that contains only 0s and 1s.

Examples:



Campusmonk

Input: nums = [1, 1, 0, 0, 1, 1, 1, 0]

Output: 3

Explanation: The maximum consecutive 1s are present from index 4 to index 6, amounting to 3 1s

Input: nums = [0, 0, 0, 0, 0, 0, 0, 0]

Output: 0

Explanation: No 1s are present in nums, thus we return 0

27.) Left Rotate Array by One

[Link](#)

Given an integer array nums, rotate the array to the left by one.

Examples:

Input: nums = [1, 2, 3, 4, 5]

Output: [2, 3, 4, 5, 1]

Explanation: Initially, nums = [1, 2, 3, 4, 5]

Rotating once to left -> nums = [2, 3, 4, 5, 1]

Input: nums = [-1, 0, 3, 6]

Output: [0, 3, 6, -1]

Explanation: Initially, nums = [-1, 0, 3, 6]

Rotating once to left -> nums = [0, 3, 6, -1]



28.) Left Rotate Array by K Places [Link](#)

Given an integer array `nums` and a non-negative integer `k`, rotate the array to the left by `k` steps.

Examples:

Input: `nums = [1, 2, 3, 4, 5, 6]`, `k = 2`

Output: `nums = [3, 4, 5, 6, 1, 2]`

Explanation: rotate 1 step to the left: `[2, 3, 4, 5, 6, 1]`

rotate 2 steps to the left: `[3, 4, 5, 6, 1, 2]`

Input: `nums = [3, 4, 1, 5, 3, -5]`, `k = 8`

Output: `nums = [1, 5, 3, -5, 3, 4]`

29.) Move Zeros to End [Link](#)

Given an integer array `nums`, move all the 0's to the end of the array. The relative order of the other elements must remain the same. This must be done in place, without making a copy of the array.

Examples:

Input: `nums = [0, 1, 4, 0, 5, 2]`

Output: `[1, 4, 5, 2, 0, 0]`

Explanation: Both the zeroes are moved to the end and the order of the other elements stay the same

Input: `nums = [0, 0, 0, 1, 3, -2]`



Output: [1, 3, -2, 0, 0, 0]

Explanation: All 3 zeroes are moved to the end and the order of the other elements stay the same

30.) Remove duplicates from sorted array [Link](#)

Given an integer array `nums` sorted in non-decreasing order, remove all duplicates in-place so that each unique element appears only once. Return the number of unique elements in the array.

If the number of unique elements be k , then,

- Change the array `nums` such that the first k elements of `nums` contain the unique values in the order that they were present originally.
- The remaining elements, as well as the size of the array does not matter in terms of correctness.

An array sorted in non-decreasing order is an array where every element to the right of an element is either equal to or greater in value than that element.

Examples:

Input: `nums = [0, 0, 3, 3, 5, 6]`

Output: 4

Explanation: Resulting array = `[0, 3, 5, 6, _, _]`

There are 4 distinct elements in `nums` and the elements marked as `_` can have any value.

31.) Find missing number [Link](#)

Given an integer array of size n containing distinct values in the range from 0 to n (inclusive), return the only number missing from the array within this range.

Examples:

Input: `nums = [0, 2, 3, 1, 4]`

Output: 5



Campusmonk

Explanation: nums contains 0, 1, 2, 3, 4 thus leaving 5 as the only missing number in the range [0, 5]

Input: nums = [0, 1, 2, 4, 5, 6]

Output: 3

Explanation: nums contains 0, 1, 2, 4, 5, 6 thus leaving 3 as the only missing number in the range [0, 6]

32.) Union of two sorted arrays [Link](#)

Given two sorted arrays **nums1** and **nums2**, return an array that contains the **union** of these two arrays. The elements in the union must be in ascending order.

The union of two arrays is an array where all values are distinct and are present in either the first array, the second array, or both.

Examples:

Input: nums1 = [1, 2, 3, 4, 5], nums2 = [1, 2, 7]

Output: [1, 2, 3, 4, 5, 7]

Explanation: The elements 1, 2 are common to both, 3, 4, 5 are from nums1 and 7 is from nums2

Input: nums1 = [3, 4, 6, 7, 9, 9], nums2 = [1, 5, 7, 8, 8]

Output: [1, 3, 4, 5, 6, 7, 8, 9]

Explanation: The element 7 is common to both, 3, 4, 6, 9 are from nums1 and 1, 5, 8 is from nums2

33.) Intersection of two sorted arrays [Link](#)

Given two sorted arrays, nums1 and nums2, return an array containing the intersection of these two arrays. Each element in the result must appear as many times as it appears in both arrays.



The intersection of two arrays is an array where all values are present in both arrays.

Examples:

Input: nums1 = [1, 2, 2, 3, 5], nums2 = [1, 2, 7]

Output: [1, 2]

Explanation: The elements 1, 2 are the only elements present in both nums1 and nums2

Input: nums1 = [1, 2, 2, 3], nums2 = [4, 5, 7]

Output: []

Explanation: No elements appear in both nums1 and nums2

34.) Leaders in an Array

[Link](#)

Given an integer array nums, return a list of all the leaders in the array.

A leader in an array is an element whose value is strictly greater than all elements to its right in the given array. The rightmost element is always a leader. The elements in the leader array must appear in the order they appear in the nums array.

Examples:

Input: nums = [1, 2, 5, 3, 1, 2]

Output: [5, 3, 2]

Explanation: 2 is the rightmost element, 3 is the largest element in the index range [3, 5], 5 is the largest element in the index range [2, 5]

Input: nums = [-3, 4, 5, 1, -4, -5]

Output: [5, 1, -4, -5]

Explanation: -5 is the rightmost element, -4 is the largest element in the index range [4, 5], 1 is the largest element in the index range [3, 5] and 5 is the largest element in the range [2, 5]

35.) Rearrange array elements by sign

[Link](#)



Given an integer array `nums` of even length consisting of an equal number of positive and negative integers. Return the answer array in such a way that the given conditions are met:

- Every consecutive pair of integers have opposite signs.
- For all integers with the same sign, the order in which they were present in `nums` is preserved.
- The rearranged array begins with a positive integer.

Examples:

Input : `nums = [2, 4, 5, -1, -3, -4]`

Output : `[2, -1, 4, -3, 5, -4]`

Explanation: The positive number 2, 4, 5 maintain their relative positions and -1, -3, -4 maintain their relative positions

Input : `nums = [1, -1, -3, -4, 2, 3]`

Output : `[1, -1, 2, -3, 3, -4]`

Explanation: The positive number 1, 2, 3 maintain their relative positions and -1, -3, -4 maintain their relative positions

36.) Print the matrix in spiral manner [Link](#)

Given an $M * N$ matrix, print the elements in a clockwise spiral manner. Return an array with the elements in the order of their appearance when printed in a spiral manner.

Examples:

Input: `matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

Output: `[1, 2, 3, 6, 9, 8, 7, 4, 5]`

Explanation: The elements in the spiral order are 1, 2, 3 -> 6, 9 -> 8, 7 -> 4, 5

Input: `matrix = [[1, 2, 3, 4], [5, 6, 7, 8]]`

Output: `[1, 2, 3, 4, 8, 7, 6, 5]`

Explanation: The elements in the spiral order are 1, 2, 3, 4 -> 8, 7, 6, 5



37.) Pascal's Triangle I [Link](#)

Given two integers r and c , return the value at the r^{th} row and c^{th} column (1-indexed) in a Pascal's Triangle.

In Pascal's triangle:

- The first row has one element with a value of 1.
- Each row has one more element in it than its previous row.
- The value of each element is equal to the sum of the elements directly above it when arranged in a triangle format.

Examples:

Input: $r = 4, c = 2$

Output: 3

Explanation: The Pascal's Triangle is as follows:

1

1 1

1 2 1

1 3 3 1

....

Thus, value at row 4 and column 2 = 3

Input: $r = 5, c = 3$

Output: 6

Explanation: The Pascal's Triangle is as follows:

1

1 1

1 2 1



1 3 3 1

1 4 6 4 1

....

Thus, value at row 5 and column 3 = 6

38.) Rotate matrix by 90 degrees [Link](#)

Given an $N * N$ 2D integer matrix, rotate the matrix by 90 degrees clockwise.

The rotation must be done in place, meaning the input 2D matrix must be modified directly.

Examples:

Input: matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Output: matrix = [[7, 4, 1], [8, 5, 2], [9, 6, 3]]

39.) Two Sum [Link](#)

Given an array of integers nums and an integer target. Return the indices(0 - indexed) of two elements in nums such that they add up to target.

Each input will have exactly one solution, and the same element cannot be used twice.

Return the answer in non-decreasing order.

Examples:

Input: nums = [1, 6, 2, 10, 3], target = 7

Output: [0, 1]

Explanation: $\text{nums}[0] + \text{nums}[1] = 1 + 6 = 7$

Input: nums = [1, 3, 5, -7, 6, -3], target = 0

Output: [1, 5]

Explanation: $\text{nums}[1] + \text{nums}[5] = 3 + (-3) = 0$



40.) 3 Sum

[Link](#)

Given an integer array `nums`. Return all triplets such that:

- $i \neq j, i \neq k, \text{ and } j \neq k$
- $\text{nums}[i] + \text{nums}[j] + \text{nums}[k] == 0$.

Notice that the solution set must not contain duplicate triplets. One element can be a part of multiple triplets. The output and the triplets can be returned in any order.

Examples:

Input: `nums = [2, -2, 0, 3, -3, 5]`

Output: `[[-2, 0, 2], [-3, -2, 5], [-3, 0, 3]]`

Explanation: $\text{nums}[1] + \text{nums}[2] + \text{nums}[0] = 0$

$\text{nums}[4] + \text{nums}[1] + \text{nums}[5] = 0$

$\text{nums}[4] + \text{nums}[2] + \text{nums}[3] = 0$

Input: `nums = [2, -1, -1, 3, -1]`

Output: `[[-1, -1, 2]]`

Explanation: $\text{nums}[1] + \text{nums}[2] + \text{nums}[0] = 0$

Note that we have used two -1s as they are separate elements with different indexes

But we have not used the -1 at index 4 as that would create a duplicate triplet

41.) Sort an array of 0's 1's and 2's

[Link](#)



Given an array **nums** consisting of only 0, 1, or 2. Sort the array in non-decreasing order. The sorting must be done in-place, without making a copy of the original array.

Examples:

Input: nums = [1, 0, 2, 1, 0]

Output: [0, 0, 1, 1, 2]

Explanation: The nums array in sorted order has 2 zeroes, 2 ones and 1 two

Input: nums = [0, 0, 1, 1, 1]

Output: [0, 0, 1, 1, 1]

Explanation: The nums array in sorted order has 2 zeroes, 3 ones and zero twos

42.) Kadane's Algorithm [Link](#)

Given an integer array **nums**, find the subarray with the largest sum and return the sum of the elements present in that subarray.

A subarray is a contiguous non-empty sequence of elements within an array.

Examples:

Input: nums = [2, 3, 5, -2, 7, -4]

Output: 15

Explanation: The subarray from index 0 to index 4 has the largest sum = 15

Input: nums = [-2, -3, -7, -2, -10, -4]

Output: -2

Explanation: The element on index 0 or index 3 make up the largest sum when taken as a subarray

43.) Next Permutation [Link](#)

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.



For example, for `arr = [1,2,3]`, the following are all the permutations of `arr`:

`[1,2,3]`, `[1,3,2]`, `[2,1,3]`, `[2,3,1]`, `[3,1,2]`, `[3,2,1]`.

The next permutation of an array of integers is the next lexicographically greater permutation of its integers.

More formally, if all the permutations of the array are sorted in lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted order.

If such arrangement is not possible (i.e., the array is the last permutation), then rearrange it to the lowest possible order (i.e., sorted in ascending order).

You must rearrange the numbers in-place and use only constant extra memory.

Examples:

Input: `nums = [1,2,3]`

Output: `[1,3,2]`

Explanation: The next permutation of `[1,2,3]` is `[1,3,2]`.

44.) Majority Element-I

[Link](#)

Given an integer array **nums** of size **n**, return the majority element of the array.

The majority element of an array is an element that appears more than $n/2$ times in the array. The array is guaranteed to have a majority element.

Examples:

Input: `nums = [7, 0, 0, 1, 7, 7, 2, 7, 7]`

Output: 7

Explanation: The number 7 appears 5 times in the 9 sized array

Input: `nums = [1, 1, 1, 2, 1, 2]`

Output: 1

Explanation: The number 1 appears 4 times in the 6 sized array

45.) Majority Element-II

[Link](#)



Campusmonk

Given an integer array **nums** of size n . Return all elements which appear more than $n/3$ times in the array. The output can be returned in any order.

Examples:

Input: **nums** = [1, 2, 1, 1, 3, 2]

Output: [1]

Explanation: Here, $n / 3 = 6 / 3 = 2$.

Therefore the elements appearing 3 or more times is : [1]

Input: **nums** = [1, 2, 1, 1, 3, 2, 2]

Output: [1, 2]

Explanation: Here, $n / 3 = 7 / 3 = 2$.

Therefore the elements appearing 3 or more times is : [1, 2]

46.) Find the repeating and missing number [link](#)

Given an integer array **nums** of size **n** containing values from [1, n] and **each** value appears **exactly** once in the array, except for **A**, which appears **twice** and **B** which is **missing**.

Return the values **A** and **B**, as an array of size 2, where **A** appears in the **0-th** index and **B** in the **1st** index.

Examples:

Input: **nums** = [3, 5, 4, 1, 1]

Output: [1, 2]

Explanation: 1 appears two times in the array and 2 is missing from **nums**



Campusmonk

Input: nums = [1, 2, 3, 6, 7, 5, 7]

Output: [7, 4]

Explanation: 7 appears two times in the array and 4 is missing from nums.

47.) Maximum Product Subarray in an Array [Link](#)

Given an integer array nums. Find the subarray with the largest product, and return the product of the elements present in that subarray.

A subarray is a contiguous non-empty sequence of elements within an array.

Examples:

Input: nums = [4, 5, 3, 7, 1, 2]

Output: 840

Explanation: The largest product is given by the whole array itself

Input: nums = [-5, 0, -2]

Output: 0

Explanation: The largest product is achieved with the following subarrays [0], [-5, 0], [0, -2], [-5, 0, -2].

48.) Merge two sorted arrays without extra space [Link](#)

Given two integer arrays **nums1** and **nums2**. Both arrays are sorted in **non-decreasing** order.

Merge both the arrays into a **single** array **sorted** in **non-decreasing** order.

- The **final** sorted array should be stored inside the array **nums1** and it should be done in-place.
- **nums1** has a length of **m + n**, where the first **m** elements denote the elements of **nums1** and rest are **0s**.



- **nums2** has a length of **n**.

Examples:

Input: nums1 = [-5, -2, 4, 5], nums2 = [-3, 1, 8]

Output: [-5, -3, -2, 1, 4, 5, 8]

Explanation: The merged array is: [-5, -3, -2, 1, 4, 5, 8], where [-5, -2, 4, 5] are from nums1 and [-3, 1, 8] are from nums2

Input: nums1 = [0, 2, 7, 8], nums2 = [-7, -3, -1]

Output: [-7, -3, -1, 0, 2, 7, 8]

Explanation: The merged array is: [-7, -3, -1, 0, 2, 7, 8], where [0, 2, 7, 8] are from nums1 and [-7, -3, -1] are from nums2

49.) Longest Consecutive Sequence in an Array [Link](#)

Given an array **nums** of **n** integers, return the length of the **longest sequence** of consecutive integers. The integers in this sequence can appear in any order.

Examples:

Input: nums = [100, 4, 200, 1, 3, 2]

Output: 4

Explanation: The longest sequence of consecutive elements in the array is [1, 2, 3, 4], which has a length of 4. This sequence can be formed regardless of the initial order of the elements in the array.

Input: nums = [0, 3, 7, 2, 5, 8, 4, 6, 0, 1]

Output: 9

Explanation: The longest sequence of consecutive elements in the array is [0, 1, 2, 3, 4, 5, 6, 7, 8], which has a length of 9.

50.) Longest subarray with sum K [Link](#)

Given an array **nums** of size **n** and an integer **k**, find the length of the **longest sub-array** that sums to **k**. If no such sub-array exists, return 0.

Examples:



Campusmonk

Input: nums = [10, 5, 2, 7, 1, 9], k=15

Output: 4

Explanation: The longest sub-array with a sum equal to 15 is [5, 2, 7, 1], which has a length of 4. This sub-array starts at index 1 and ends at index 4, and the sum of its elements (5 + 2 + 7 + 1) equals 15. Therefore, the length of this sub-array is 4.

Input: nums = [-3, 2, 1], k=6

Output: 0

Explanation: There is no sub-array in the array that sums to 6. Therefore, the output is 0.

51.) Count subarrays with given sum [Link](#)

Given an array of integers nums and an integer k, return the total **number of subarrays** whose sum equals to k.

Examples:

Input: nums = [1, 1, 1], k = 2

Output: 2

Explanation: In the given array [1, 1, 1], there are two subarrays that sum up to 2: [1, 1] and [1, 1]. Hence, the output is 2.

Input: nums = [1, 2, 3], k = 3

Output: 2

Explanation: In the given array [1, 2, 3], there are two subarrays that sum up to 3: [1, 2] and [3]. Hence, the output is 2.

52.) Maximum Points You Can Obtain from Cards [Link](#)



Campusmonk

Given N cards arranged in a row, each card has an associated score denoted by the cardScore array. Choose exactly k cards. In each step, a card can be chosen either from the beginning or the end of the row. The score is the sum of the scores of the chosen cards.

Return the **maximum score** that can be obtained.

Examples:

Input : cardScore = [1, 2, 3, 4, 5, 6] , k = 3

Output : 15

Explanation : Choosing the rightmost cards will maximize your total score. So optimal cards chosen are the rightmost three cards 4 , 5 , 6.

Th score is $4 + 5 + 6 \Rightarrow 15$.

Input : cardScore = [5, 4, 1, 8, 7, 1, 3] , k = 3

Output : 12

Explanation : In first step we will choose card from beginning with score of 5.

In second step we will choose the card from beginning again with score of 4.

In third step we will choose the card from end with score of 3.

The total score is $5 + 4 + 3 \Rightarrow 12$

53.) Reverse a String II [Link](#)

Given a **string**, the task is to **reverse** it. The string is represented by an array of characters **s**. Perform the reversal in place with **O(1) extra memory**.

Examples:

Input : s = ["h", "e", "l", "l", "o"]

Output : ["o", "l", "l", "e", "h"]



Campusmonk

Explanation : The given string is `s = "hello"` and after reversing it becomes `s = "olleh"`.

Input : `s = ["b", "y", "e"]`

Output : `["e", "y", "b"]`

Explanation : The given string is `s = "bye"` and after reversing it becomes `s = "eyb"`.

54.) Palindrome Check

[Link](#)

You are given a string `s`. Return **true** if the string is palindrome, otherwise **false**. A string is called **palindrome** if it reads the same forward and backward.

Examples:

Input : `s = "hannah"`

Output : `true`

Explanation : The given string when read backward is `-> "hannah"`, which is same as when read forward.

Hence answer is true.

Input : `s = "aabbbaa"`

Output : `false`

Explanation : The given string when read backward is `-> "aabbbaa"`, which is not same as when read forward.

Hence answer is false.

55.) Largest Odd Number in a String

[Link](#)

Given a string `s`, representing a large integer, the task is to return the largest-valued **odd integer** (as a string) that is a **substring** of the given string `s`.



The number returned should **not** have leading **zero's**. But the given input string may have leading zero.

Examples:

Input : s = "5347"

Output : "5347"

Explanation : The odd numbers formed by given strings are --> 5, 3, 53, 347, 5347.

So the largest among all the possible odd numbers for given string is 5347.

Input : s = "0214638"

Output : "21463"

Explanation : The different odd numbers that can be formed by the given string are --> 1, 3, 21, 63, 463, 1463, 21463.

We cannot include 021463 as the number contains leading zero.

So largest odd number in given string is 21463.

56.) Longest Common Prefix [Link](#)

Write a function to find the **longest common prefix** string amongst an array of strings.

If there is no common prefix, return an empty string "".

Examples:

Input : str = ["flowers", "flow", "fly", "flight"]

Output : "fl"

Explanation : All strings given in array contains common prefix "fl".

Input : str = ["dog", "cat", "animal", "monkey"]

Output : ""

Explanation : There is no common prefix among the given strings in array.



57.) Isomorphic String

[Link](#)

Given two strings **s** and **t**, determine if they are **isomorphic**. Two strings **s** and **t** are **isomorphic** if the characters in **s** can be replaced to get **t**.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Examples:

Input : **s** = "egg", **t** = "add"

Output : true

Explanation : The 'e' in string **s** can be replaced with 'a' of string **t**.

The 'g' in string **s** can be replaced with 'd' of **t**.

Hence all characters in **s** can be replaced to get **t**.

58.) Rotate String

[Link](#)

Given two strings **s** and **goal**, return **true** if and only if **s** can become **goal** after some number of shifts on **s**.

A **shift** on **s** consists of moving the leftmost character of **s** to the rightmost position.

For example, if **s** = "abcde", then it will be "bcdea" after one shift.

Examples:

Input : **s** = "abcde", **goal** = "cdeab"

Output : true

Explanation : After performing 2 shifts we can achieve the goal string from string **s**.

After first shift the string **s** is => bcdea

After second shift the string **s** is => cdeab.



59.) Valid Anagram

[Link](#)

Given two strings **s** and **t**, return **true** if **t** is an anagram of **s**, and **false** otherwise.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Examples:

Input : s = "anagram" , t = "nagaram"

Output : true

Explanation : We can rearrange the characters of string **s** to get string **t** as frequency of all characters from both strings is same.

Input : s = "dog" , t = "cat"

Output : false

Explanation : We cannot rearrange the characters of string **s** to get string **t** as frequency of all characters from both strings is not same.

60.) Sort Characters by Frequency

[Link](#)

You are given a string **s**. Return the array of **unique** characters, sorted by **highest to lowest** occurring characters.

If two or more characters have same frequency then arrange them in alphabetic order.

Examples:

Input : s = "tree"

Output : ['e', 'r', 't']

Explanation : The occurrences of each character are as shown below :

e --> 2



Campusmonk

r --> 1

t --> 1.

The r and t have same occurrences , so we arrange them by alphabetic order.

61.) Longest Substring Without Repeating Characters [Link](#)

Given a string, S. Find the **length** of the longest substring without repeating characters.

Examples:

Input : S = "abccddabac"

Output : 4

Explanation : The answer is "abcd" , with a length of 4.

Input : S = "aaabbbcccc"

Output : 2

Explanation : The answers are "ab" , "bc". Both have maximum length 2.

62.) Longest Substring With At Most K Distinct Characters

[Link](#)

Given a string s and an integer k. Find the length of the **longest substring** with at most k distinct characters.

Examples:

Input : s = "aababbcaacc" , k = 2

Output : 6

Explanation : The longest substring with at most two distinct characters is "aababb".

The length of the string 6.

Input : s = "abcddefg" , k = 3



Output : 4

Explanation : The longest substring with at most three distinct characters is "bcdd".

The length of the string 4.

63.) Minimum Window Substring [Link](#)

Given two strings s and t. Find the **smallest** window substring of s that includes all characters in t (including duplicates) , in the window. Return the empty string "" if no such substring exists.

Examples:

Input : s = "ADOBECODEBANC" , t = "ABC"

Output : "BANC"

Explanation : The minimum window substring of string s that contains the string t is "BANC".

Input : s = "a" , t = "a"

Output : "a"

Explanation : The complete string is the minimum window

64.) Number of Substrings Containing All Three Characters

[Link](#)

Given a string s , consisting only of characters 'a' , 'b' , 'c'. Find the **number** of substrings that contain at least **one** occurrence of all these characters 'a' , 'b' , 'c'.

Examples:

Input : s = "abcba"

Output : 5

Explanation : The substrings containing at least one occurrence of the characters 'a' , 'b' , 'c' are "abc" , "abcb" , "abcba" , "bcba" , "cba".



Campusmonk

Input : s = "ccabcc"

Output : 8

Explanation : The substrings containing at least one occurrence of the characters 'a' , 'b' , 'c' are "ccab" , "ccabc" , "ccabcc" , "cab" , "cabcc" , "cabcc" , "abc" , "abcc".

PYQ -> Story based

Q1. Problem Statement –

Given a string S (input consisting) of '*' and '#'. The length of the string is variable. The task is to find the minimum number of '*' or '#' to make it a valid string. The string is considered valid if the number of '*' and '#' are equal. The '*' and '#' can be at any position in the string.

Note: The output will be a positive or negative integer based on number of '*' and '#' in the input string.

(*>#): positive integer



Campusmonk

(#>): negative integer

(#=*): 0

Example 1:

Input 1: ###*** -> Value of S

Output : 0 → number of * and # are equal

Q2. Given an integer array Arr of size N the task is to find the count of elements whose value is greater than all of its prior elements.

Note : 1st element of the array should be considered in the count of the result. For example, Arr[]={7,4,8,2,9} As 7 is the first element, it will consider in the result. 8 and 9 are also the elements that are greater than all of its previous elements. Since total of 3 elements is present in the array that meets the condition.

Hence the output = 3.

Example 1:

Input 5 -> Value of N, represents size of Arr

7-> Value of Arr[0]

4 -> Value of Arr[1]

8-> Value of Arr[2]

2-> Value of Arr[3]

9-> Value of Arr[4]

Output : 3

Q3. At a fun fair, a street vendor is selling different colours of balloons. He sells N number of different colours of balloons (B[]). The task is to find the colour (odd) of the balloon which is present odd number of times in the bunch of balloons.

Note: If there is more than one colour which is odd in number, then the first colour in the array which is present odd number of times is displayed. The colours of the balloons can all



Campusmonk

be either upper case or lower case in the array. If all the inputs are even in number, display the message "All are even".

Example 1: 7 -> Value of N [r,g,b,b,g,y,y] -> B[] Elements B[0] to B[N-1], where each input element is separated by new line.

Output : r -> [r,g,b,b,g,y,y] -> "r" colour balloon is present odd number of times in the bunch.

Explanation:

From the input array above: r: 1 balloon g: 2 balloons b: 2 balloons y : 2 balloons Hence , r is only the balloon which is odd in number.

Q4. You are given two integer arrays nums1 and nums2, sorted in non-decreasing order, and two integers m and n,

representing the number of elements in nums1 and nums2 respectively.

Merge nums1 and nums2 into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array nums1. To

accommodate this, nums1 has a length of m + n, where the first m elements denote the elements that should be

merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n.

Example 1:

Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3

Output: [1,2,2,3,5,6]

Explanation: The arrays we are merging are [1,2,3] and [2,5,6].

The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.



Campusmonk

Q5. A party has been organized on cruise. The party is organized for a limited time(T). The number of guests

entering ($E[i]$) and leaving ($L[i]$) the party at every hour is represented as elements of the array. The task is to

find the maximum number of guests present on the cruise at any given instance within T hours.

Example 1:

Input :

5 -> Value of T

[7,0,5,1,3] -> $E[]$, Element of $E[0]$ to $E[N-1]$, where input each element is separated by new line

[1,2,1,3,4] -> $L[]$, Element of $L[0]$ to $L[N-1]$, while input each element is separate by new line.

Output :

8 -> Maximum number of guests on cruise at an instance.

Q6. There are total n number of Monkeys sitting on the branches of a huge Tree. As travelers offer Bananas and Peanuts, the

Monkeys jump down the Tree. If every Monkey can eat k Bananas and j Peanuts. If total m number of Bananas and p number

of Peanuts are offered by travelers, calculate how many Monkeys remain on the Tree after some of them jumped down to eat.

At a time one Monkey gets down and finishes eating and go to the other side of the road. The Monkey who climbed down

does not climb up again after eating until the other Monkeys finish eating.

Monkey can either eat k Bananas or j Peanuts. If for last Monkey there are less than k Bananas left on the ground or less than j



Campusmonk

Peanuts left on the ground, only that Monkey can eat Bananas(<k) along with the Peanuts(<j).

Write code to take inputs as n, m, p, k, j and return the number of Monkeys left on the Tree.

Where, n= Total no of Monkeys

k= Number of eatable Bananas by Single Monkey (Monkey that jumped down last may get less than k Bananas)

j = Number of eatable Peanuts by single Monkey (Monkey that jumped down last may get less than j Peanuts)

m = Total number of Bananas

p = Total number of Peanuts

Remember that the Monkeys always eat Bananas and Peanuts, so there is no possibility of k and j having a value zero.

Q7. The Caesar cipher is a type of substitution cipher in which each alphabet in the plaintext or messages is shifted by a number of places down the alphabet.

For example, with a shift of 1, P would be replaced by Q, Q would become R, and so on. To pass an encrypted message from one person to another, it is first necessary that both parties have the 'Key' for the cipher, so that the sender may encrypt and the receiver may decrypt it. Key is the number of OFFSET to shift the cipher alphabet. Key can have basic shifts from 1 to 25 positions as there are 26 total alphabets.

As we are designing custom Caesar Cipher, in addition to alphabets, we are considering numeric digits from 0 to 9. Digits can also be shifted by key places.

For Example, if a given plain text contains any digit with values 5 and key =2, then 5 will be replaced by 7, "-" (minus sign) will remain as it is.

Key value less than 0 should result into "INVALID INPUT"

Example 1: Enter your Plaintext: All the best

Enter the Key: 1

The encrypted Text is: Bmm uif Cftu



Campusmonk

Q8. Sort one array according to another array You are given two arrays $a[]$ (integer) and $b[]$ (char). The i th value of $a[]$ corresponds to the i th value of $b[]$.

Sort the array $b[]$ with respect to $a[]$.

Note: The output is whitespace and newline character sensitive. After every character print a whitespace character. Also do not print any newline character at any point.

Example 1: Input: $a[] = \{3, 1, 2\}$ $b[] = \{'G', 'E', 'K'\}$

Output: E K G

Explanation: Here 3 corresponds to G, 1 corresponds to 'E', 2 corresponds to 'K'

Q9. There are total n number of Monkeys sitting on the branches of a huge Tree. As travelers offer Bananas and Peanuts, the

Monkeys jump down the Tree. If every Monkey can eat k Bananas and j Peanuts. If total m number of Bananas and p number

of Peanuts are offered by travelers, calculate how many Monkeys remain on the Tree after some of them jumped down to eat.

At a time one Monkey gets down and finishes eating and go to the other side of the road. The Monkey who climbed down

does not climb up again after eating until the other Monkeys finish eating.

Monkey can either eat k Bananas or j Peanuts. If for last Monkey there are less than k Bananas left on the ground or less than j

Peanuts left on the ground, only that Monkey can eat Bananas($<k$) along with the Peanuts($<j$).



Campusmonk

Write code to take inputs as n, m, p, k, j and return the number of Monkeys left on the Tree.

Where, n= Total no of Monkeys

k= Number of eatable Bananas by Single Monkey (Monkey that jumped down last may get less than k Bananas)

j = Number of eatable Peanuts by single Monkey(Monkey that jumped down last may get less than j Peanuts)

m = Total number of Bananas

p = Total number of Peanuts

Remember that the Monkeys always eat Bananas and Peanuts, so there is no possibility of k and j having a value zero.

Example 1:

Input Values -> 20 2 3 12 12

Output Values-> Number of Monkeys left on the tree:10

Note: Kindly follow the order of inputs as n,k,j,m,p as given in the above example. And output must include the same format as in above example(Number of Monkeys left on the Tree:)

For any wrong input display INVALID INPUT

Q10. Airport security officials have confiscated several item of the passengers at the security check point. All the items

have been dumped into a huge box (array). Each item possesses a certain amount of risk[0,1,2]. Here, the risk severity of

the items represent an array[] of N number of integer values. The task here is to sort the items based on their levels of

risk in the array. The risk values range from 0 to 2.

Example:



Campusmonk

Input:

7 -> Value of N

[1,0,2,0,1,0,2]-> Element of arr[0] to arr[N-1], while input each element is separated by new line.

Output :

0 0 0 1 1 2 2 -> Element after sorting based on risk severity

Q11. You are given an array prices where prices[i] is the price of a given stock on the ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: prices = [7,1,5,3,6,4]

Output: 5

Q12. Jack is always excited about sunday. It is favourite day, when he gets to play all day. And goes to

cycling with his friends.

So every time when the months starts he counts the number of sundays he will get to enjoy. Considering

the month can start with any day, be it Sunday, Monday.... Or so on.



Campusmonk

Count the number of Sunday jack will get within n number of days.

Example 1:

Input

mon-> input String denoting the start of the month.

13 -> input integer denoting the number of days from the start of the month.

Output :

2 -> number of days within 13 days.

Q13. There is an integer array 'A' of size 'N'.

A sequence is successive when the adjacent elements of the sequence have a difference of 1.

You must return the length of the longest successive sequence.

For example,

Input:

A = [5, 8, 3, 2, 1, 4], N = 6

Output: 5

Q14. Problem Description -: In this 3 Palindrome, Given an input string word, split the string into exactly 3 palindromic substrings. Working from left to right, choose the smallest split for the first substring that still allows the remaining word to be split into 2 palindromes.



Campusmonk

Similarly, choose the smallest second palindromic substring that leaves a third palindromic substring.

If there is no way to split the word into exactly three palindromic substrings, print "Impossible" (without quotes). Every character of the string needs to be consumed.

Cases not allowed –

After finding 3 palindromes using above instructions, if any character of the original string remains unconsumed.

No character may be shared in forming 3 palindromes.

Constraints

$1 \leq \text{the length of input string} \leq 1000$

Input

First line contains the input string consisting of characters between [a-z].

Output

Print 3 substrings one on each line.

Time Limit

1

Examples

Input -> nayannamantenet

Output->

nayan

naman

telnet

Q15. Problem Statement – An automobile company manufactures both a two wheeler (TW) and a four wheeler (FW). A company manager wants to make the production of both types of vehicle according to the given data below:

1st data, Total number of vehicle (two-wheeler + four-wheeler)=v

2nd data, Total number of wheels = W



Campusmonk

The task is to find how many two-wheelers as well as four-wheelers need to manufacture as per the given data.

Example :

Input :

200 -> Value of V

540 -> Value of W

Output :

TW =130 FW=70

Explanation:

$130 + 70 = 200$ vehicles

$(70 * 4) + (130 * 2) = 540$ wheels

Constraints :

$2 \leq W$

$W \% 2 = 0$

$V < W$

Print "INVALID INPUT" , if inputs did not meet the constraints.

The input format for testing

The candidate has to write the code to accept two positive numbers separated by a new line.

First Input line – Accept value of V.

Second Input line- Accept value for W.

Q16. A parking lot in a mall has $R \times C$ number of parking spaces. Each parking space will either be empty(0) or full(1). The status (0/1) of a parking space is represented as the element of the matrix. The task is to find index of the prpeinzta row(R) in the parking lot that has the most of the parking spaces full(1).



Campusmonk

Note :

RxC- Size of the matrix

Elements of the matrix M should be only 0 or 1.

Example 1:**Input :**

3 -> Value of R(row)

3 -> value of C(column)

[0 1 0 1 1 0 1 1 1] -> Elements of the array M[R][C] where each element is separated by new line.

Output :

3 -> Row 3 has maximum number of 1's

Q17. A washing machine works on the principle of Fuzzy System, the weight of clothes put inside it for washing is uncertain But based on weight measured by sensors, it decides time and water level which can be changed by menus given on the machine control area.

For low level water, the time estimate is 25 minutes, where approximately weight is between 2000 grams or any nonzero positive number below that.

For medium level water, the time estimate is 35 minutes, where approximately weight is between 2001 grams and 4000 grams.

For high level water, the time estimate is 45 minutes, where approximately weight is above 4000 grams.

Assume the capacity of machine is maximum 7000 grams

Where approximately weight is zero, time estimate is 0 minutes.

Write a function which takes a numeric weight in the range [0,7000] as input and produces estimated time as output is: "OVERLOADED", and for all other inputs, the output statement is

"INVALID INPUT".

Input should be in the form of integer value –



Campusmonk

Output must have the following format –

Time Estimated: Minutes

Example:

Input value

2000

Output value

Time Estimated: 25 minutes

Q18.

Problem Statement

A doctor has a clinic where he serves his patients. The doctor's consultation fees are different for different groups of patients depending on their age. If the patient's age is below 17, fees is 200 INR. If the patient's age is between 17 and 40, fees is 400 INR. If patient's age is above 40, fees is 300 INR. Write a code to calculate earnings in a day for which one array/List of values representing age of patients visited on that day is passed as input.

Note:

- Age should not be zero or less than zero or above 120
- Doctor consults a maximum of 20 patients a day
- Enter age value (press Enter without a value to stop):

Example 1:

Input

20

30

40

50

2

3

14

Output

Total Income 2000 INR

Note: Input and Output Format should be same as given in the above example.

For any wrong input display INVALID INPUT



Campusmonk

Output Format

Total Income 2100 INR

Q19. Joseph is learning digital logic subject which will be for his next semester. He usually tries to solve unit assignment problems before the lecture. Today he got one tricky question. The problem statement is “A positive integer has been given as an input. Convert decimal value to binary representation. Toggle all bits of it after the most significant bit including the most significant bit. Print the positive integer value after toggling all bits”.

Constraints-

$1 \leq N \leq 100$

Example 1:**Input :**

10 -> Integer

Output :

5 -> result- Integer

Explanation:

Binary representation of 10 is 1010. After toggling the bits(1010), will get 0101 which represents “5”. Hence output will print “5”.

Q20. A supermarket maintains a pricing format for all its products. A value N is printed on each product. When the scanner reads the value N on the item, the product of all the digits in the value N is the price of the item. The task here is to design the software such that given the code of any item N the product (multiplication) of all the digits of value should be computed(price).

Example 1:



Campusmonk

Input :

5244 -> Value of N

Output :

160 -> Price

Explanation:

From the input above

Product of the digits 5,2,4,4

$$5*2*4*4 = 160$$

Hence, output is 160.

Q21. A furnishing company is manufacturing a new collection of curtains. The curtains are of two colors aqua(a) and black (b). The curtains color is represented as a string(str) consisting of a's and b's of length N. Then, they are packed (substring) into L number of curtains in each box. The box with the maximum number of 'aqua' (a) color curtains is labeled. The task here is to find the number of 'aqua' color curtains in the labeled box.

Note :

If 'L' is not a multiple of N, the remaining number of curtains should be considered as a substring too. In simple words, after dividing the curtains in sets of 'L', any curtains left will be another set(refer example 1)

Example 1:**Input :**

bbbaaababa -> Value of str

3 -> Value of L

Output:



3 -> Maximum number of a's

Explanation:

From the input given above.

Dividing the string into sets of 3 characters each

Set 1: {b,b,b}

Set 2: {a,a,a}

Set 3: {b,a,b}

Set 4: {a} -> leftover characters also as taken as another set

Among all the sets, Set 2 has more number of a's. The number of a's in set 2 is 3.

Hence, the output is 3.

Q22. An international round table conference will be held in india. Presidents from all over the world representing their respective countries will be attending the conference. The task is to find the possible number of ways(P) to make the N members sit around the circular table such that.

The president and prime minister of India will always sit next to each other.

Example 1:

Input :

4 -> Value of N(No. of members)

Output :

12 -> Possible ways of seating the members

Explanation:

2 members should always be next to each other.



So, 2 members can be in $2!$ ways

Rest of the members can be arranged in $(4-1)!$ ways. (1 is subtracted because the previously selected two members will be considered as single members now).

So total possible ways 4 members can be seated around the circular table $2 \times 6 = 12$.

Hence, output is 12.

Example 2:

Input:

10 -> Value of N (No. of members)

Output :

725760 -> Possible ways of seating the members

Explanation:

2 members should always be next to each other.

So, 2 members can be in $2!$ ways

Rest of the members can be arranged in $(10-1)!$ Ways. (1 is subtracted because the previously selected two members will be considered as a single member now).

So, total possible ways 10 members can be seated around a round table is

$2 \times 362880 = 725760$ ways.

Hence, output is 725760.

Q23. An array is considered special if every pair of its adjacent elements contains two numbers with different parity.



Campusmonk

You are given an array of integers `nums`. Return true if `nums` is a special array, otherwise, return false.

Example 1:

Input: `nums = [1]`

Output: true

Explanation:

There is only one element. So, the answer is true.

Example 2:

Input: `nums = [2,1,4]`

Output: true

Explanation:

There is only two pairs: (2,1) and (1,4), and both of them contain numbers with different parity. So the answer is true.

Example 3:

Input: `nums = [4,3,1,6]`

Output: false

Explanation:

`nums[1]` and `nums[2]` are both odd. So, the answer is false.

Q24) Given an array of integers and a sum, the task is to count all subsets of given array with sum equal to given sum.

Input :

The first line of input contains an integer `T` denoting the number of test cases. Then `T` test cases follow. Each test case contains an integer `n` denoting the size of the array. The next line contains `n` space separated integers forming the array. The last line contains the sum.

Output :

Count all the subsets of given array with sum equal to given sum.

NOTE: Since result can be very large, print the value modulo 10^9+7 .

Constraints :

$1 \leq T \leq 100$



Campusmonk

$1 \leq n \leq 103$

$1 \leq a[i] \leq 103$

$1 \leq \text{sum} \leq 103$

Example :

Input :

2

6

2 3 5 6 8 10

10

5

1 2 3 4 5

10

Output :

3

3

Explanation :

Testcase 1: possible subsets : (2,3,5) , (2,8) and (10)

Testcase 2: possible subsets : (1,2,3,4) , (2,3,5) and (1,4,5)

Q25) Before the outbreak of corona virus to the world, a meeting happened in a room in Wuhan. A person who attended that meeting had COVID-19 and no one in the room knew about it! So everyone started shaking hands with everyone else in the room as a gesture of respect and after meeting unfortunately every one got infected! Given the fact that any two persons shake hand exactly once, Can you tell the total count of handshakes happened in that meeting?



Campusmonk

Input Format :

The first line contains the number of test cases T, T lines follow.

Each line then contains an integer N, the total number of people attended that meeting.

Output Format :

Print the number of handshakes for each test-case in a new line.

Constraints :

$1 \leq T \leq 1000$

$0 < N < 10^6$

Sample Input :

2

1

2

Output :

0

1

Explanation :

Case 1 : The lonely board member shakes no hands, hence 0.

Case 2 : There are 2 board members, 1 handshake takes place.

Q26) For enhancing the book reading, the school distributed story books to students as part of the Children's Day celebrations. To increase the reading habit, the class teacher decided to exchange the books every week so that everyone will have a different book to read. She wants to know how many possible exchanges are possible.



Campusmonk

If they have 4 books and students, the possible exchanges are 9. Bi is the book of i-th student and after the exchange, he should get a different book, other than Bi.

B1 B2 B3 B4 – first state, before exchange of the books

B2 B1 B4 B3

B2 B3 B4 B1

B2 B4 B1 B3

B3 B1 B4 B2

B3 B4 B1 B2

B3 B4 B2 B1

B4 B1 B2 B3

B4 B3 B1 B2

B4 B3 B2 B1

Find the number of possible exchanges, if the books are exchanged so that every student will receive a different book.

Constraints

$1 \leq N \leq 1000000$

Input Format

Input contains one line with N, indicates the number of books and number of students.

Output Format

Output the answer modulo 100000007.

Refer the sample output for formatting

Sample Input :

4



Sample Output :

9

Q27) You are given a string A and you have to find the number of different sub-strings of the string A which are fake palindromes.

Note:

1. Palindrome: A string is called a palindrome if you reverse the string yet the order of letters remains the same. For example, MADAM.
2. Fake Palindrome: A string is called as a fake palindrome if any of its permutations is a palindrome. For example, AAC is fake palindrome, but ACD is not.
3. Sub-string: A sub-string is a contiguous sequence (non-empty) of characters within a string.
4. Two sub-strings are considered same if their starting indices and ending indices are equal.

Input Format:

First line contains a string S

Output Format:

Print a single integer (number of fake palindrome sub-strings).

Constraints:

$1 \leq |S| \leq 2 * 10^5$

The string will contain only Upper case 'A' to 'Z'

Sample Input 1:

ABAB

Sample Output 1:

7

Explanation:



The fake palindrome for the string ABAB are A, B, A, B, ABA, BAB, ABAB.

Sample Input 2:

AAA

Sample output 2:

6

Explanation:

The fake palindrome for the string AAA are A, A, A, AA, AA, AAA.

Q 28. Problem Statement:

You are given a binary tree with N nodes where each node contains a positive integer value. The goal is to find the *maximum sum* path from the root to any leaf node. However, you need to ensure that the sum of the values along the path is *not* divisible by a given integer K .

Input:

- The first line of input contains an integer N , the number of nodes in the tree.
- The second line contains N space-separated integers representing the values of the nodes.
- The third line contains $N-1$ pairs of integers u and v , representing the edges of the tree (the tree is rooted at node 1).
- The fourth line contains a single integer K , the divisor.

Output:

- Print the maximum sum of the path from the root to any leaf node such that the sum is not divisible by K . If no such path exists, print -1.

Input:

7 3 4 8 2 1 6 10 1 2 1 3 2 4 2 5 3 6 3 7 5

Output:

21

Explanation:

- The possible paths from the root (node 1) to the leaves are:



Campusmonk

- 3 -> 4 -> 2 (sum = 9)
- 3 -> 4 -> 1 (sum = 8)
- 3 -> 8 -> 6 (sum = 17)
- 3 -> 8 -> 10 (sum = 21)
- The path with the maximum sum that is not divisible by 5 is 3 -> 8 -> 10, with a sum of 21.

Notes :

- Consider edge cases where no valid path exists.
- Optimize the solution for large inputs where N can be as large as 100,000.

CampusMonk



Campusmonk

Linked List:

Crud on LL: -

- Traversal in Linked List
- Deletion in Linked List
- Insertion in Linked List
- Deletion of the head of LL
- Deletion of the tail of LL
- Deletion of the Kth element of LL
- Delete the element with value X
- Insertion at the head of LL
- Insertion at the tail of LL
- Insertion at the Kth position of LL

Doubly LinkedList: -

- Creation of doubly linkedlist
- Deletion in Doubly LL
- Insertion in DLL
- Convert Array to DLL
- Delete head of DLL
- Delete Tail of DLL
- Delete Kth Element of DLL
- Insert node before head in DLL



Campusmonk

- Insert node before tail in DLL

Logical Ques on LL: -

- Reverse a LL
- Find Middle of Linked List
- Delete the middle node in LL
- Check if LL is palindrome or not
- Find the intersection point of Y LL
- Detect a loop in LL

[link](#)[link](#)[link](#)[link](#)[link](#)[link](#)

Stack & Queue: -

- Valid Parenthesis
- Next Greater Element
- Asteroid Collision
- Trapping Rainwater

[link](#)[link](#)[link](#)[link](#)

Greedy Problems: -

Assign cookies

[link](#)

Lemonade Change

[link](#)

Jump game 1

[link](#)



Campusmonk

CampusMonk